

USEEN1 --- Software Supply Chain Security for the Cloud

CNAM Paris

Build-time SBOM

Comparison & Analysis

Mini Project 1 — Submission Report

Author: Ali Raza

Course: USEEN1 — Software Supply Chain Security

Institution: CNAM Paris

Repository: github.com/alirazahub/mp1-buildtime-sbom

Contents

1	Project Overview	2
1.1	Motivation	2
1.2	Scope	2
2	Project Objectives	2
3	Selected OSS Projects	2
3.1	Node.js Application	3
3.2	Python Flask API	3
3.3	Go Gin Application	3
4	Tools and Technologies	3
4.1	Syft (Anchore)	3
4.2	Trivy (Aqua Security)	4
4.3	Witness (TestifySec)	4
4.4	Supporting Tools	5
5	Methodology	5
5.1	Workflow Overview	5
5.2	Scan Levels	5
5.3	Commands Executed	5
5.4	OS Dependency Exclusion	6
6	Results	6
6.1	Application-Level Dependency Counts (OS Excluded)	6
6.2	Raw Component Counts (Including OS)	7
6.3	Notable Observations	7
7	Key Findings	7
7.1	Missing Dependencies	7
7.2	Unexpected Components	8
7.3	Build/Runtime Mismatches	8
8	Challenges and Decisions	9
8.1	Witness Output Format vs CycloneDX	9
8.2	OS Dependency Inflation	9
8.3	Python Virtual Environment	9
8.4	Syft and Trivy Divergence at Source	9
8.5	Go SBOM Underrepresentation at Image Level	10
9	Conclusions	10
10	Next Steps and Recommendations	10
11	Repository Structure	11
	References	12

1. Project Overview

This report documents the design, execution, and findings of Mini Project 1 for USEEN1 at CNAM Paris. The project investigates Software Bill of Materials (SBOM) generation at different stages of the software supply chain, with the goal of understanding the differences in coverage, granularity, and accuracy between source-level and image-level SBOM generation approaches.

A Software Bill of Materials is a formal, structured record of the components, libraries, and dependencies that make up a software artifact. SBOMs are central to supply chain security because they make the composition of software auditable — enabling vulnerability tracking, license compliance, and provenance verification.

1.1. Motivation

The core question driving this project is: *does what a developer declares in their source code accurately reflect what actually runs in production?* The consistent finding across all three projects evaluated is: not even close. Source-level SBOMs represent only the application layer as declared in package manifests, while deployed containers carry an entire runtime environment that is invisible to source scanners.

1.2. Scope

The project covers:

- Source-level SBOM generation using Syft and Trivy
- Container image-level SBOM generation using Syft and Trivy
- Build-time attestation using Witness
- Comparison across three OSS projects in different language ecosystems
- Analysis of missing dependencies, unexpected components, and tool divergence

2. Project Objectives

1. **Evaluate SBOM Levels** — Compare SBOMs generated at source and container image levels across three OSS projects with different packaging behaviors.
2. **Identify Gaps** — Detect missing dependencies, unexpected runtime components, and build/runtime mismatches between scan levels.
3. **Measure Granularity** — Analyze the depth of component metadata — version data, license information, hash values, PURLs, CPEs — across Syft and Trivy.
4. **Understand Tool Divergence** — Explain why Syft and Trivy produce different results from the same artifact, and determine which is more appropriate for which security purpose.

3. Selected OSS Projects

Three open-source projects were selected to span different language ecosystems and packaging behaviors. This was intentional: ecosystem differences reveal how the source-to-image gap varies by language, compilation model, and runtime environment.

3.1. Node.js Application

Repository: [docker-node-example](#) (nickjj)
Language: JavaScript / Node.js
Package Manager: npm / `package.json`
Purpose: Express-based web backend service

npm is the largest package registry in the world and is well-known for producing deep transitive dependency trees. A single declared dependency can pull in dozens of sub-dependencies, making it the most interesting ecosystem for source-vs-image comparison.

3.2. Python Flask API

Repository: [sample-flask-api-restx](#) (app-generator)
Language: Python
Package Manager: pip / `requirements.txt`
Purpose: REST API service

Python's packaging behavior differs significantly from npm. Source declarations are typically minimal, but container images introduce a full CPython runtime, pip toolchain, and additional libraries not referenced in `requirements.txt`. An important methodology note: the Python project was scanned without an isolated virtual environment (`venv`), which means source-level scans may include system-wide Python packages beyond what `requirements.txt` strictly declares. This is a limitation documented under Section 8.

3.3. Go Gin Application

Repository: [go-gin-example](#) (eddyjy)
Language: Go
Package Manager: Go modules / `go.mod`
Purpose: Lightweight HTTP REST API

Go's static compilation model produces a unique SBOM challenge. All dependencies declared in `go.mod` are compiled into a single binary at build time. Image-level scanning sees fewer individual packages than source-level scanning because the binary does not expose its constituent modules as separate artifacts. This makes Go the clearest demonstration of why source-level SBOMs can be more informative than image-level SBOMs for certain languages.

4. Tools and Technologies

4.1. Syft (Anchore)

Syft is an open-source SBOM generation tool developed by Anchore. It generates SBOMs by physically crawling files, binaries, and metadata within a directory or container image layer-by-layer. Rather than trusting what package managers declare, Syft fingerprints what is actually present on disk.

Approach: Binary fingerprinting — opens every file, reads metadata, identifies packages by content signatures.

Strength: Finds packages not declared in any manifest, including vendored libraries and hand-copied binaries.

Output format: CycloneDX 1.6 JSON (`.cdx.json`)

4.2. Trivy (Aqua Security)

Trivy is an open-source vulnerability and SBOM scanner developed by Aqua Security. It generates SBOMs by querying the package manager databases and manifest files within the scanned artifact.

Approach: Manifest-oriented resolution — reads `package.json`, `requirements.txt`, `go.mod`, and OS package databases.

Strength: Fast, clean, accurate against declared dependencies.

Output format: CycloneDX 1.6 JSON (`.cdx.json`)

Why Syft and Trivy Always Diverge

Syft and Trivy use fundamentally different data sources. Syft builds its own picture by crawling the filesystem. Trivy queries the package manager’s official record. These two sources never fully agree because:

- Packages installed without a package manager are visible to Syft but absent from Trivy’s database
- Nested or vendored dependencies are counted individually by Syft but as one entry by Trivy
- Both tools apply different classification rules to decide what is an “application package” vs an “OS package”
- Go is an exception where Trivy finds more at source level because `go.sum` explicitly lists all indirect dependencies

The divergence is not a defect — it is a characteristic. Using both tools together provides more complete coverage than either alone.

4.3. Witness (TestifySec)

Witness is a supply chain security tool that generates cryptographically signed attestations of build processes. It wraps a build command and records what executed, what files were accessed, and what the build produced, storing the result as a signed JSON envelope.

Predicate type: <https://witness.testifysec.com/attestation-collection/v0.1>

Output: Signed JSON attestation envelopes stored in `attestations/`

Keys: Build signing key materials stored in `keys/`

Scope note: In this project, Witness was used to attest the `docker` build process, producing build provenance. A true runtime SBOM would require wrapping the *executing application* to observe runtime library loads and system calls. Build-time attestation is a valid and valuable use case for Witness, but is distinct from runtime SBOM generation. This distinction is documented as a limitation in Section 8.

4.4. Supporting Tools

Tool	Purpose	Usage
Docker	Container build	Build Alpine-based images for all three projects
CycloneDX 1.6	SBOM format standard	Output format for all Syft and Trivy scans
jq	JSON parsing	Extract component counts, filter OS packages
Shell scripts	Automation	Batch scanning and comparison across projects

5. Methodology

5.1. Workflow Overview

The experiment followed a six-step pipeline for each of the three OSS projects:

1. Clone the OSS project repository
2. Run source-level scan with Syft and Trivy on the application directory
3. Build a Docker container image from the project's Dockerfile
4. Run image-level scan with Syft and Trivy on the built image
5. Generate a build attestation using Witness wrapping the `docker build` command
6. Compare SBOM outputs across tools and scan levels

5.2. Scan Levels

Layer 1 — Source Level

Source-level scans analyze the application source directory, reading manifest files (`package.json`, `requirements.txt`, `go.mod`) and installed package artifacts. This represents the *declared* dependency state — what the developer explicitly included.

Layer 2 — Container Image Level

Image-level scans analyze all layers of a built Docker container. This includes the application dependencies plus the base OS environment, runtime interpreter, system libraries, and any tooling installed during the image build. This represents the *deployed* state — what actually runs in production.

Layer 3 — Build Attestation

Witness wraps the `docker build` command and produces a signed JSON attestation recording the build process. This provides cryptographic proof of build integrity and provenance.

5.3. Commands Executed

```
# Syft - source scan
syft dir:.. -o cyclonedx-json > sboms/[app]-syft-source.cdx.json
```

```
# Syft - image scan
syft image:<tag> -o cyclonedx-json > sboms/[app]-syft-image.
  cdx.json

# Trivy - source scan
trivy fs . --format cyclonedx > sboms/[app]-trivy-source.cdx.
  json

# Trivy - image scan
trivy image <tag> --format cyclonedx > sboms/[app]-trivy-image
  .cdx.json

# Witness - build attestation
witness run -s build --signer-file-key-path keys/buildkey.pem
  \
  -- docker build -t <tag> .

# Component count extraction
jq '.components | length' sbom.cdx.json

# OS package exclusion filter (pkg:apk for Alpine)
jq '[.components[] | select(.purl | startswith("pkg:apk") |
  not)
  | select(.purl | startswith("pkg:deb") | not)] | length'
  sbom.cdx.json
```

5.4. OS Dependency Exclusion

Per the project brief, OS-level dependencies were excluded from the primary comparison. All Docker images were built on Alpine Linux base images. Alpine packages use the `pkg:apk` PURL namespace. The `jq` filter above was applied to all image-level SBOMs to isolate application-layer components only.

This exclusion is important because OS packages dominate raw image-level counts — for example, the Python image contains over 22,000 raw components with Syft, but only 51 application-layer packages after OS exclusion. Comparing raw image counts against source scans would be misleading because the source scan never includes OS packages in the first place.

6. Results

6.1. Application-Level Dependency Counts (OS Excluded)

Table 1 presents the component counts after excluding OS packages from image-level scans, providing a clean application-layer comparison.

Table 1: SBOM Component Counts — Application Dependencies (OS Packages Excluded)

Project	Syft Source	Syft Image	Trivy Source	Trivy Image	Syft Δ
Node.js	902	1,319	273	1,418	+46%
Python (Flask)	8	51	9	520	+538%
Go (Gin)	107	57	231	267	-47%

6.2. Raw Component Counts (Including OS)

For reference, Table 2 presents the unfiltered counts. These numbers include Alpine OS packages and are provided for completeness only — they are not the basis for comparison per the project brief.

Table 2: Raw SBOM Component Counts (Including OS Packages)

Project	Syft Source	Syft Image	Trivy Source	Trivy Image
Node.js	906	5,337	273	1,418
Python (Flask)	9	22,236	9	520
Go (Gin)	121	13,912	231	267

6.3. Notable Observations

Go -47%: Go is the only project where image-level scanning finds *fewer* application packages than source-level scanning. This is because static compilation merges all `go.mod` dependencies into a single binary. Syft cannot decompose the binary into its constituent modules, so image-level counts drop. This is the most revealing finding of the project — it shows that for statically compiled languages, source-level SBOMs are more informative than image-level SBOMs.

Python +538%: After OS exclusion, Python still shows a large application-layer gap. The image introduces a full CPython runtime, pip toolchain, and additional runtime libraries not declared in `requirements.txt`. This confirms that Python’s packaging model genuinely introduces hidden application-layer dependencies at the container level, beyond just OS noise.

Node.js +46%: The relatively modest increase reflects npm’s behaviour at install time — some transitive dependencies are installed that are not declared in `package.json` directly. The container adds relatively few extras beyond what npm installs, because Node.js applications carry most of their dependencies in `node_modules/`.

7. Key Findings

The project brief specified three comparison criteria. Each is addressed below.

7.1. Missing Dependencies

Source-level SBOMs consistently undercount the real application-layer footprint:

- **Python:** 8 source deps → 51 image deps. The image introduces 43 additional application-layer packages invisible in `requirements.txt`, including the CPython runtime and pip toolchain.
- **Node.js:** 902 source deps → 1,319 image deps. npm installs extra transitive dependencies at build time not explicitly declared in `package.json`.
- **Go:** Unique case — source declares 107 modules but image scan reveals only 57 fingerprintable components. The remaining modules are compiled into the binary and are genuinely missing from image-level scanning.

7.2. Unexpected Components

Components present in the image but absent from source declarations:

- **All projects:** Alpine Linux base image injects OS-level packages not declared in any application manifest. These were excluded from primary analysis per the brief.
- **Python:** Full CPython interpreter, pip, setuptools, and wheel are present in the image but never mentioned in `requirements.txt`.
- **Node.js:** Node.js runtime binary and npm toolchain appear in the image without declaration in `package.json`.
- **Go:** Minimal unexpected components at application level — static compilation and Alpine base keep the footprint clean. This makes Go containers the most predictable from a security audit perspective.

7.3. Build/Runtime Mismatches

Syft and Trivy produce systematically different results from identical artifacts:

- **At source level:** Node.js shows 902 (Syft) vs 273 (Trivy) — a 3.3× difference. Syft physically walks `node_modules/` and counts everything installed; Trivy reads only `package.json` declarations.
- **At image level:** Node.js shows 1,319 (Syft) vs 1,418 (Trivy). The tools converge somewhat because both are scanning a fully installed environment.
- **Go at source:** Trivy finds *more* than Syft (231 vs 107). `go.sum` explicitly lists all indirect dependencies, giving Trivy a richer manifest to read than Syft's binary analysis yields.
- **Python image:** Trivy finds 520 vs Syft's 51 (after OS exclusion). Trivy reads the full Python package database including toolchain; Syft's application-filter is more conservative.

Core conclusion on tool divergence: Neither Syft nor Trivy is definitively correct. They answer different questions. Syft answers: *“what is physically present?”* Trivy answers: *“what was officially installed?”* The divergence is a finding in itself — it demonstrates that relying on a single SBOM scanner gives an incomplete picture of software composition, and that combining both tools provides more robust supply chain visibility.

8. Challenges and Decisions

8.1. Witness Output Format vs CycloneDX

Challenge: Witness produces signed JSON attestation envelopes with a custom predicate type (`witness.testifysec.com/v0.1`), not CycloneDX component lists. Directly integrating Witness output into the jq-based component count comparison pipeline requires custom parsing of the attestation payload.

Decision: Use Witness for the build provenance layer and Syft/Trivy for component comparison. Attestation JSON artifacts are stored in `attestations/` and signing keys in `keys/`. Parsing Witness attestations for component-level comparison is documented as a next step.

Limitation acknowledged: A true runtime SBOM requires wrapping the *executing application*, not the build process. Build-time attestation records build integrity; runtime observation records what actually loads at execution time. The correct next step is to wrap the running container with Witness and compare the runtime observation against the source SBOM.

8.2. OS Dependency Inflation

Challenge: The project brief explicitly states not to focus on OS dependencies. Raw image-level counts are dominated by OS packages — Python reaches 22,236 components with Syft before filtering, of which only 51 are application-layer packages.

Decision: Use Alpine-based Docker images intentionally throughout, as Alpine’s minimal footprint (150-300 OS packages) significantly reduces OS noise compared to Debian (600+ packages). Apply jq PURL filters (`pkg:apk`) post-scan to exclude OS packages from all primary comparisons.

8.3. Python Virtual Environment

Challenge: The Python Flask project was scanned without an isolated virtual environment. This means source-level scans may include system-wide Python packages installed on the host machine, not just what `requirements.txt` declares.

Decision: Document as a methodology limitation. The correct approach is to create a `venv`, install only the declared requirements, and scan within that isolated environment to get a clean application-only baseline.

Impact: The Python source count (8-9 packages) appears approximately correct because `requirements.txt` was minimal. However, in a production methodology this isolation step is mandatory.

8.4. Syft and Trivy Divergence at Source

Challenge: Syft and Trivy produce significantly different counts from the same source directory (Node.js: 902 vs 273). This initially appeared to be a tool defect.

Decision: After investigation, confirmed that this is expected behaviour reflecting different resolution strategies. Report both counts and explain the difference. Treat image-level

counts as the more reliable comparison point because both tools have access to the fully materialised artifact.

8.5. Go SBOM Underrepresentation at Image Level

Challenge: Go’s image scan shows fewer components than its source scan (-47%), which is counterintuitive. This appears to contradict the general finding that image-level SBOMs reveal more than source-level.

Decision: Identified as a language-specific characteristic of static compilation, not a scanning error. Documented as a key finding demonstrating that the source-to-image relationship is not universally “more at image level” — it depends on the compilation model.

9. Conclusions

This project demonstrated that source-level SBOMs are a necessary but insufficient basis for supply chain security. The key conclusions are:

1. **Source SBOMs undercount real footprint.** Across all three projects, source-level scanning misses components that are present in the deployed container. For Python, the application-layer gap is +538%; for Node.js, +46%.
2. **Image-level SBOMs are the closest proxy for deployed reality.** Image scans capture what actually runs in production, including runtime dependencies not declared by the developer. They should be the standard for security audits.
3. **Go is the exception.** Static compilation inverts the expected relationship — source SBOMs capture more granular dependency information than image scans for statically compiled languages. This makes source-level scanning more important, not less, for Go.
4. **Tool choice matters and divergence is a finding.** Syft and Trivy consistently produce different counts. The divergence reflects genuine differences in resolution strategy, not error. Using both tools provides complementary coverage that neither provides alone.
5. **Build attestation adds a provenance layer.** Witness attestations provide cryptographic proof that the build process was observed and recorded. Combined with SBOM generation, this enables end-to-end supply chain traceability from source to deployed artifact.
6. **OS dependency exclusion is essential for meaningful comparison.** Raw image counts are dominated by OS packages that inflate the apparent source-to-image gap. Filtering to application-layer components is a prerequisite for actionable comparison.

10. Next Steps and Recommendations

1. **Filter OS packages systematically.** Integrate PURL-based OS exclusion into the scan pipeline at generation time using Trivy’s `--vuln-type library` flag, rather than as a post-processing step.

2. **Implement true runtime SBOM with Witness.** Wrap the executing application (not the build) with Witness to observe which libraries are loaded, which system calls are made, and which files are accessed at runtime. Compare this against the source SBOM to produce the most accurate source-to-runtime gap analysis.
3. **Isolate Python environment.** Re-run the Python project scans within a `venv` to eliminate system-wide package contamination from source-level results.
4. **Vulnerability analysis.** Run all generated SBOMs through Grype or OSV to compare how many CVEs are detected per scan level and per tool. This would quantify the security risk of relying on source-only SBOMs.
5. **SBOM diff tooling.** Use `sbomasm` or `cdxgen diff` to automate the detection of missing, added, and changed components between scan levels, replacing manual jq comparisons.
6. **SLSA alignment.** Align Witness build attestations with SLSA Level 2/3 provenance requirements, enabling the repository to serve as a reference implementation for SLSA-compliant build provenance.
7. **Expand to more projects.** The current three-project dataset is sufficient to establish patterns but limited for statistical confidence. Expanding to more projects per language ecosystem would strengthen the generalisability of the findings.

11. Repository Structure

```

mp1-buildtime-sbom/
  apps/
    nodejs-app/          # Express Node.js
  application
    python-flask-app/    # Flask REST API
  application
    go-app/              # Go Gin HTTP service
  sboms/
    nodejs-syft-source.cdx.json
    nodejs-syft-image.cdx.json
    nodejs-trivy-source.cdx.json
    nodejs-trivy-image.cdx.json
    python-syft-source.cdx.json
    python-syft-image.cdx.json
    python-trivy-source.cdx.json
    python-trivy-image.cdx.json
    go-syft-source.cdx.json
    go-syft-image.cdx.json
    go-trivy-source.cdx.json
    go-trivy-image.cdx.json
  attestations/
    nodejs-build-attestation.json
    python-build-attestation.json
    go-build-attestation.json
  keys/
    buildkey.pem         # Private signing key

```

```

    buildpublic.pem      # Public verification key
results/
    nodejs/              # Raw scan output for
Node.js
    python/              # Raw scan output for
Python
    go/                  # Raw scan output for Go
summary.txt             # Aggregated counts
comparison
    README.md

```

References

- Anchore. *Syft* — CLI tool and Go library for generating a Software Bill of Materials from container images and filesystems. <https://github.com/anchore/syft>
- Aqua Security. *Trivy* — Find vulnerabilities, misconfigurations, secrets, SBOM in containers, Kubernetes, code repositories, clouds and more. <https://github.com/aquasecurity/trivy>
- TestifySec. *Witness* — A tool for supply chain security. <https://github.com/in-toto/witness>
- CycloneDX. *CycloneDX* — A full-stack Bill of Materials (BOM) standard. <https://cyclonedx.org>
- CISA / NTIA. *The Minimum Elements For a Software Bill of Materials (SBOM)*. <https://www.cisa.gov/sbom>
- OpenSSF. *SLSA* — Supply-chain Levels for Software Artifacts. <https://slsa.dev>
- nickjj. *docker-node-example*. <https://github.com/nickjj/docker-node-example>
- app-generator. *sample-flask-api-restx*. <https://github.com/app-generator/sample-flask-api>
- eddycjy. *go-gin-example*. <https://github.com/eddcjy/go-gin-example>